

CAX Scan Procedures

This document is a quick list of procedures needed to efficiently perform mirror, slits and caustic scans in the Carcará-X beamline.

Before the scanning

These are essentially steps to ensure the beam is available and the scripts are working properly.

1. Manually "find the beam"

- Be sure the beamline shutters are open by checking the HMI frontend;
- Open the slit GUI (at the terminal, run: `launch_cax_gui.sh slit1/2`) and check the slits are open;
- Open the DVF1 & 2 GUIs (at the terminal, `launch_cax_gui.sh dvf`); check the 'log' (logarithm) box to help finding the beam;
- Run `launch_cax_gui.sh mirror`, adjust mirror motor positions; if needed, check last recorded positions before maintenance (archiver: `CAX:A:PB01:m*.RBV`);

2. Run `<path>/cax-scripts/caxmirrorscan-cli` once to check if all PVs are being accessed correctly (the machine state is printed out);

3. (Important) Calibration of the slits span and the DVF image size. The relation between the slit motor span and the DVF1 size is calculated from the YAG dimensions (named `WSIZE_H` and `WSIZE_V`) and motor positions to properly identify the image suitable areas. Check if the hard-coded values converting pixel intervals seen by the DVF to slit motor position are still correct:

- Found in `caxmirrorscan-cli` (functions named `_{top, bottom, left, right}_pos_from_pixels`);
- It can be done by using the DVFA and SlitA GUIs:
 - measure total YAG crystal size in pixels (`WSIZE_V` and `WSIZE_H`);
 - close each blade individually, taking note of initial position and total displacement;
 - update these values in the code.
- Update `Config` class attributes in `<path>/cax-scripts/caxscripts/config.py`.

Obs.: Hopefully, these changes remain permanent or self-adjustable in the near future.

Data acquisition

Once the previous section was accomplished, the scanning should be straightforward.

1. Run `<path>/cax-scripts/caxmirrorscan-cli` and follow the menu directives:

1. Decide what variable to scan (e.g: TX, RY etc);
2. Choose the number of scans (a full scan may be repeated \$n\$ times);
3. Define the initial and final position/angle of a scan;
4. Define how many points to measure in each scan;
5. Choose a suitable file location and name convention: e.g: `mirror_tx_passXX....`

After each scan, the motors should turn back to their initial positions, according to the previously registered machine state.

Quick Analysis

Once one or more passes of different scans are saved, following a consistent naming convention, a quick analysis can be carried out by accessing the `<path>/cax-scripts/scan_analysis.ipynb` Jupyter notebook:

1. Run **cell 01** to import necessary modules;
2. Adjust **cell 02** providing:
 - `workdir`: where files are stored;
 - `file_pattern`: regular expression matching naming convention.
3. **cell 03** contains main entry to plot observables:
 - Add desired observables to `observables` variable;
 - Main observables: *centroid*, *fwhm*, *intensity*, etc;
 - Select:
 - `pass_interval`;
 - `step_interval`: in case one chooses to omit certain points in a scan;

This will result in a general plot of the observables across different passes, providing useful information to guide further scans.